

Q7) Write a C program for the following ciphertext generated using a simple substitution algorithm. 53†††305))6*;4826)4†.)4†);806*;48†8¶(60))85;;]8*;;†8783 (88)5†;46(;8896?:8) †(;485);5†2:‡(;49562(5—4)8¶8 ;4069285);)6†8)4††;1(†9;48081;8:8†1;48†85;4)485†528806*81 (†9;48;(88;4(†?34;48)4†;161;;188;†?; Decrypt this message.

1. As you know, the most frequently occurring letter in English is e. Therefore, the first or second (or perhaps third?) The most common character in the message is likely to stand for e. Also, e is often seen in pairs (e.g., meet, fleet, speed, seen, been, agree, etc.). Try to find a character in the ciphertext that decodes to e.
2. The most common word in English is “the.” Use this fact to guess the characters that stand for t and h.
3. Decipher the rest of the message by deducing additional words.

Aim: To write a C program to decrypt a ciphertext encrypted using a **Simple Substitution Cipher** by applying **frequency analysis** and identifying common English letter and word patterns.

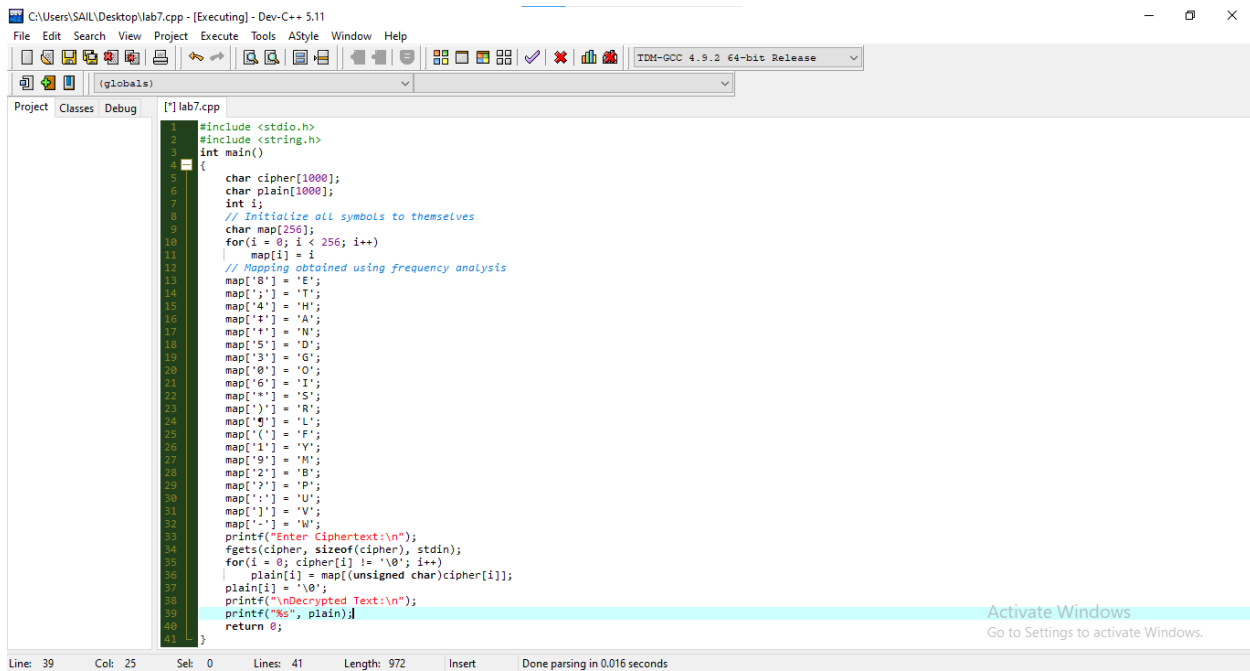
Algorithm:

1. Start the program.
2. Read the given ciphertext.
3. Count the frequency of occurrence of each symbol in the ciphertext.
4. Assume the most frequently occurring symbol represents the letter 'E', the most common letter in English.
5. Identify common patterns such as the word "**THE**" to determine additional letter mappings.
6. Substitute the identified symbols with their corresponding plaintext letters.
7. Continue deducing the remaining mappings using common English words and repeated letter patterns.
8. Replace all ciphertext symbols with the discovered plaintext letters to obtain the decrypted message.
9. Display the decrypted plaintext.
10. Stop the program.

Result:

Thus, the C program to decrypt the given **Simple Substitution Cipher** using **frequency analysis** was successfully executed, and the original plaintext message was recovered by identifying common letter frequencies and English word pattern

Code:



```
1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     char cipher[1000];
6     char plain[1000];
7     int i;
8     // Initialize all symbols to themselves
9     char map[256];
10    for(i = 0; i < 256; i++)
11    {
12        map[i] = i;
13        // Mapping obtained using frequency analysis
14        map['0'] = 'E';
15        map['1'] = 'T';
16        map['4'] = 'H';
17        map['1'] = 'A';
18        map['1'] = 'N';
19        map['5'] = 'D';
20        map['9'] = 'G';
21        map['0'] = 'O';
22        map['6'] = 'I';
23        map['*'] = 'S';
24        map['1'] = 'R';
25        map['5'] = 'L';
26        map['1'] = 'F';
27        map['1'] = 'Y';
28        map['9'] = 'M';
29        map['2'] = 'B';
30        map['2'] = 'P';
31        map['1'] = 'U';
32        map['1'] = 'V';
33        map['1'] = 'W';
34        printf("Enter Ciphertext:\n");
35        fgets(cipher, sizeof(cipher), stdin);
36        for(i = 0; cipher[i] != '\0'; i++)
37        {
38            plain[i] = map[(unsigned char)cipher[i]];
39        }
40        printf("\nDecrypted Text:\n");
41        printf("%s", plain);
42        return 0;
43    }
```

Output:



```
C:\Users\SAIL\Desktop\Lab4.exe
Enter Ciphertext: Buub
Possible Plaintexts:
a= 1 b= 0 --> BUUB
a= 1 b= 1 --> ATTA
a= 1 b= 2 --> ZSSZ
a= 1 b= 3 --> YRRY
a= 1 b= 4 --> XQQX
a= 1 b= 5 --> WPPW
a= 1 b= 6 --> VOOV
a= 1 b= 7 --> UNNU
a= 1 b= 8 --> TMMT
a= 1 b= 9 --> SLLS
a= 1 b=10 --> RKKR
a= 1 b=11 --> QJJQ
a= 1 b=12 --> PIIP
a= 1 b=13 --> OHHO
a= 1 b=14 --> NGGN
a= 1 b=15 --> MFFM
a= 1 b=16 --> LEEL
a= 1 b=17 --> KDDK
a= 1 b=18 --> JCCJ
a= 1 b=19 --> IBBI
a= 1 b=20 --> HAAH
a= 1 b=21 --> GZZG
a= 1 b=22 --> FYYF
a= 1 b=23 --> EXXE
a= 1 b=24 --> DWWD
a= 1 b=25 --> CVVC
```